

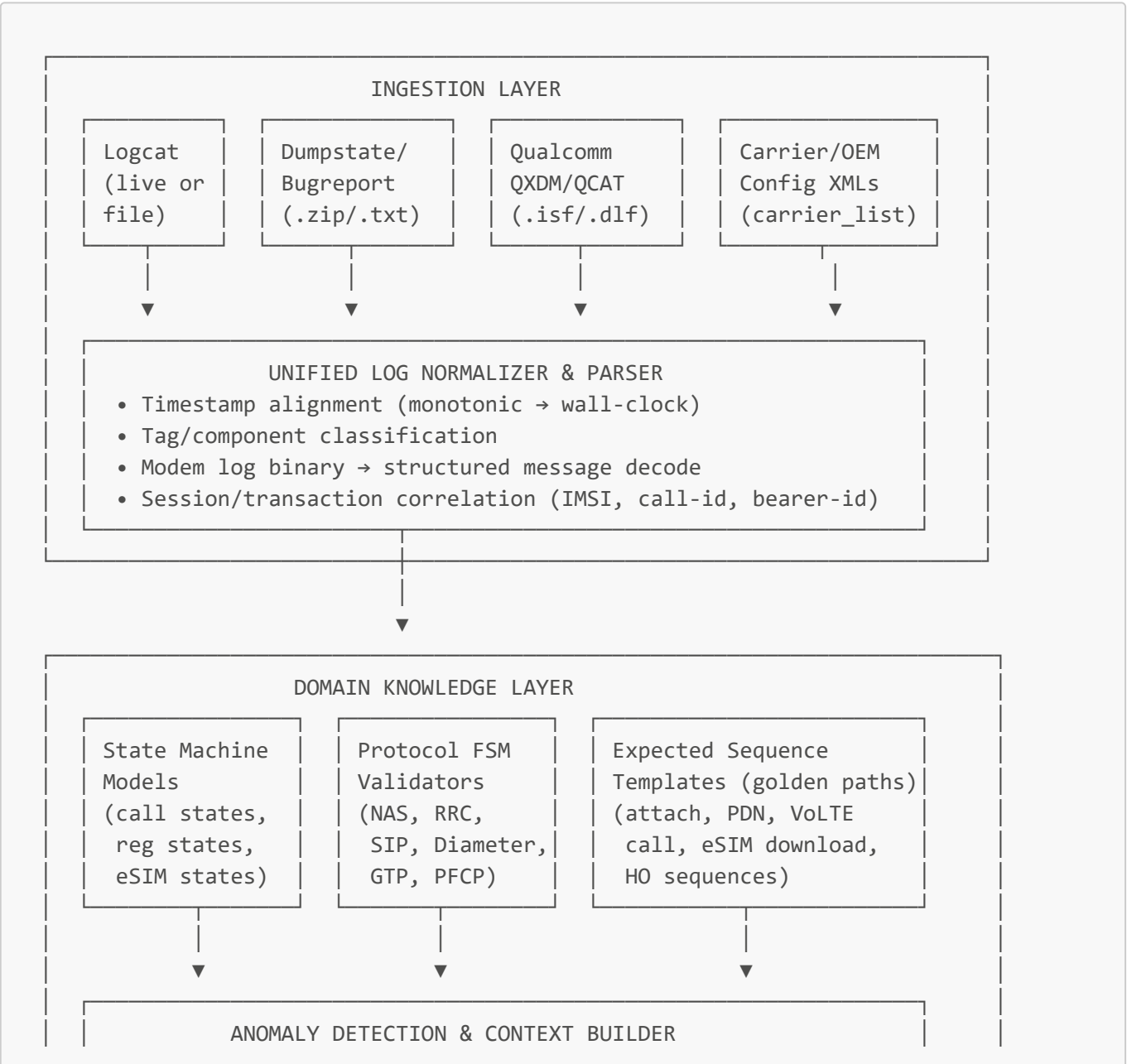
AI/LLM-Based Android Telecom Log Analysis Solution

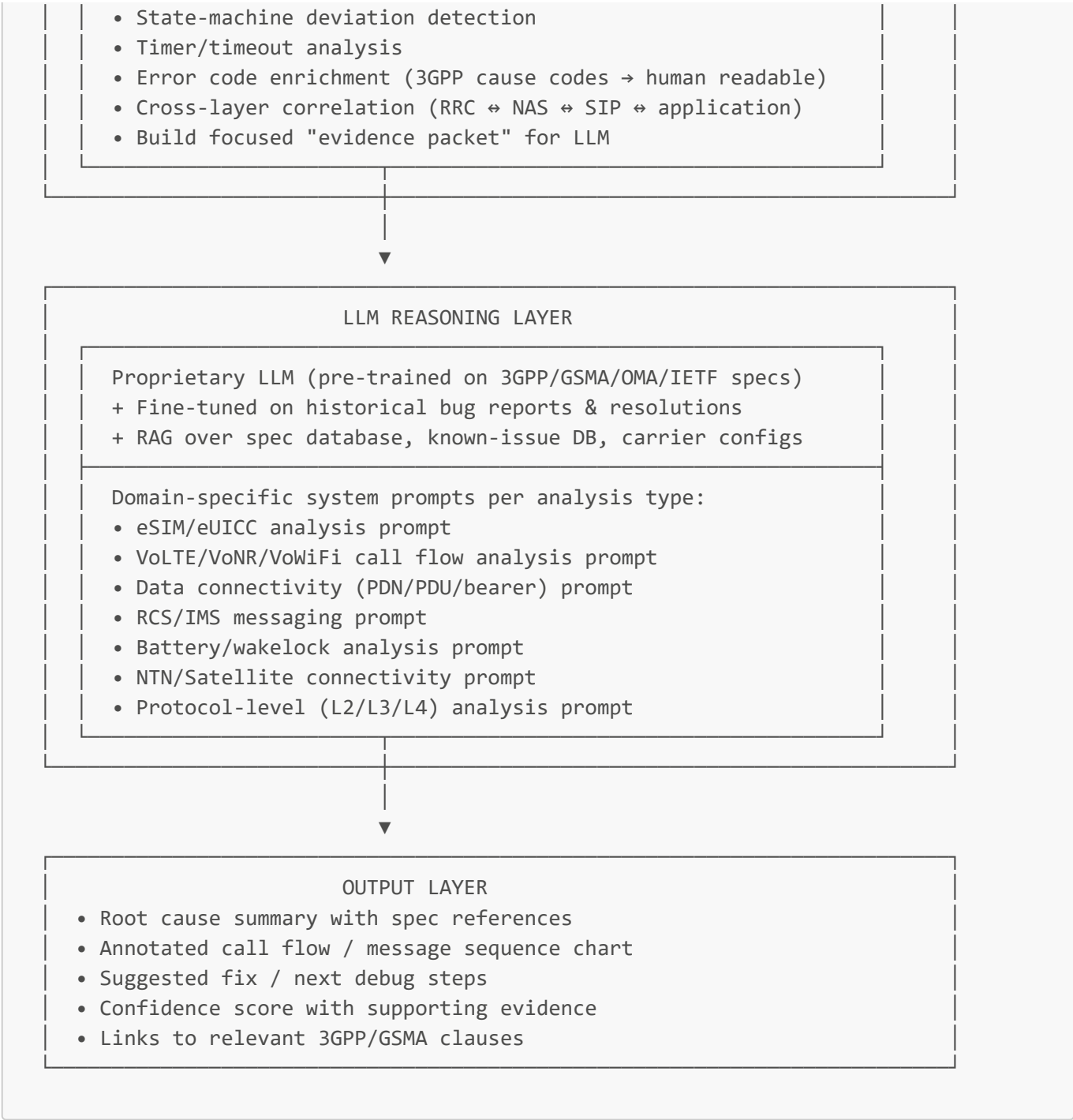
Architecture & Implementation Guide

1. Executive Summary

This document outlines the architecture for an AI/LLM-powered log analysis system purpose-built for Android telephony, modem protocol, and connectivity engineers. The system ingests Android logcat, dumpstate/bugreport, and Qualcomm modem (QXDM/QCAT) logs, normalizes and structures them through domain-specific parsers, constructs enriched context windows, and leverages a proprietary LLM (pre-trained on 3GPP/GSMA/ETSI specifications) to perform root cause analysis across eSIM provisioning, VoLTE/VoNR/VoWiFi, RCS, data connectivity, NTN satellite, battery drain, and L2-L4 protocol issues.

2. System Architecture Overview





3. Ingestion & Parsing Layer — Detailed Design

3.1 Android Logcat Parsing

Logcat is your primary application/framework-level data source. The parser must handle both standard and "long" formats.

Key tag families to extract and classify:

Domain	Critical Logcat Tags	What They Reveal
Telephony Framework	Telephony, TelephonyRegistry, PhoneInterfaceManager, GsmCdmaPhone, Phone, PhoneFactory	High-level telephony state changes

Domain	Critical Logcat Tags	What They Reveal
RIL (Radio Interface Layer)	RILJ, RILC, RIL, RILD, RilRequest, RadioResponse, RadioIndication	Commands sent to/from modem via HAL
IMS/VoLTE/VoNR	ImsPhone, ImsCall, ImsManager, ImsService, ImsConnectionStateListener, MmTelFeature, ImsCallSession, ImsRegistration	IMS registration, call setup, feature status
SIP Stack	okhttpSIP, okhttpTag (some OEMs), or vendor SIP tags	SIP INVITE/BYE/REGISTER messages
Data Connectivity	DcTracker, DataConnection, DcController, ApnContext, DataNetwork, DataNetworkController, TrafficController	APN configuration, PDN/PDU session management
eSIM/eUICC	EuiccController, EuiccService, EuiccCardManager, UiccController, UiccSlotUtil, SubscriptionController, SubscriptionManagerService	Profile download/enable/disable/switch
Network Selection	ServiceStateTracker, NetworkRegistrationInfo, CellIdentity, NetworkTypeController, AccessNetworksManager	PLMN selection, registration state, RAT
RCS/Messaging	RcsService, RcsProvider, ImsRcsManager, SipTransport, PresencePublisher	RCS registration, capability exchange
Connectivity	ConnectivityService, NetworkAgentInfo, WifiService, IWlan, EpdgTunnelManager	WiFi calling tunnel, network switching
Battery/Power	BatteryStats, PowerManagerService, AlarmManager, WakeLock, JobScheduler	Wakelock acquisition, alarm firing patterns
NTN/Satellite	SatelliteController, SatelliteModemInterface, PointingAppController, SatelliteSessionController, DatagramController	Satellite modem state, datagram TX/RX

Parser requirements:

```
For each logcat line, extract:  
- wall_clock_timestamp (ms precision)  
- PID / TID  
- log_level (V/D/I/W/E/F)  
- tag  
- message body  
- classified_domain (enum: TELEPHONY, RIL, IMS, SIP, DATA, ESIM,  
                        NETWORK_SEL, RCS, CONNECTIVITY, POWER, NTN, OTHER)  
- extracted_entities:
```

- call_id, bearer_id, cid (connection id)
- IMSI/ICCID fragments (mask sensitive digits)
- APN name
- error_code / cause_code
- RAT type (LTE, NR, IWLAN, NTN)
- SIP method + response code
- NAS message type
- RRC state

3.2 Dumpstate / Bugreport Parsing

A dumpstate zip contains dozens of critical sections. The parser must decompose it:

Priority sections to extract:

Section / File	Contents	Analysis Value
<code>dumpsys telephony.registry</code>	Current ServiceState, SignalStrength, DataConnectionState per phone	Snapshot of telephony state at capture time
<code>dumpsys isub</code> (SubscriptionManagerService)	Active subscriptions, slot mapping, eSIM group UUIDs	SIM/eSIM configuration verification
<code>dumpsys ims</code>	IMS registration state per slot, feature capabilities (VoLTE/VT/VoWiFi/RCS), MMTEL/RCS feature tags	IMS capability negotiation status
<code>dumpsys carrier_config</code>	Per-carrier config overrides (KEY_CARRIER_VOLTE_AVAILABLE, KEY_IMS_WIFI_CALLING, RTT keys, 5G keys)	Whether carrier has enabled specific features
<code>dumpsys connectivity</code>	Active networks, network capabilities, link properties, default network	Data path verification
<code>dumpsys wifi</code>	WiFi state, OFDM stats, score, connected BSSID	WiFi calling pre-conditions
<code>dumpsys batterystats</code>	UID-level wakelock durations, alarm counts, network usage, foreground time	Battery drain attribution
<code>dumpsys alarm</code>	Pending alarms per UID, wakeup alarm counts	Alarm-driven drain analysis
<code>dumpsys netpolicy</code>	Background data restrictions, app standby bucket per UID	Policy-driven connectivity issues
<code>dumpsys activity providers</code> (telephony.db, mmssms.db)	Call log, SMS/MMS DB state	Message delivery verification
<code>logcat</code> (main, radio, events, kernel)	Embedded logcat buffers — radio buffer is critical for modem-adjacent messages	Historical logcat if live wasn't captured

Section / File	Contents	Analysis Value
dmesg / kernel log	Kernel-level events: thermal throttle, modem crash, remoteproc state	Modem subsystem restart (SSR) detection
tombstones/	Native crash traces	OMACP, IMS daemon, RIL daemon crashes
vendor/ sections	OEM-specific modem logging, QMI message traces if enabled	Qualcomm-specific diagnostics

3.3 Qualcomm Modem Log Parsing (QXDM / QCAT / .isf / .dlf)

This is the deepest and most technically demanding parsing tier. Qualcomm modem logs contain binary-encoded protocol messages.

Approach: Build or wrap a decoder that can extract structured messages from these formats.

Critical Qualcomm log message IDs and what to extract:

Log Code / Message	Protocol Layer	Key Fields to Extract
0xB0C0 — LTE RRC OTA	RRC (L3)	Message type (RRCConnectionSetup, RRCConnectionReconfiguration, MeasurementReport, etc.), SRB/DRB config, measConfig, NR-related IEs (for EN-DC/NR-DC)
0xB821 — NR RRC OTA	NR RRC (L3)	RRCSetup, RRCReconfiguration, CellGroupConfig, measConfig, SRB/DRB, BWP config
0xB0EC — LTE NAS EMM OTA	NAS (L3)	Attach Request/Accept/Reject, TAU, Service Request, ESM cause codes, EPS bearer context
0xB0ED — LTE NAS ESM OTA	NAS (L3)	PDN Connectivity Request/Accept, Activate Bearer, EPS QoS, APN, TFT
0xB826 — NR NAS 5GMM OTA	5GS NAS (L3)	Registration Request/Accept/Reject, Service Request, 5GMM cause, NSSAI, LADN
0xB827 — NR NAS 5GSM OTA	5GS NAS (L3)	PDU Session Establishment/Modification/Release, QoS rules/flows, SSC mode, S-NSSAI
0xB0A2 — LTE MAC UL/DL	MAC (L2)	HARQ stats, DCI grants, BSR, PHR, scheduling metrics
0xB173 — LTE PDCP UL/DL	PDCP (L2)	SN, ROHC, integrity protection, cipher config
0xB0A1 — LTE RLC UL/DL	RLC (L2)	RLC mode (AM/UM), retransmission stats, poll/status PDUs

Log Code / Message	Protocol Layer	Key Fields to Extract
0xB111 — LTE PDSCH stats	PHY (L1)	MCS, BLER, rank, CQI, throughput
0xB8D0 — NR PDSCH stats	PHY (L1)	NR-specific MCS, rank, BLER, DL throughput
0x1568 / 0x156D — IMS SIP messages	SIP	Full SIP message: REGISTER, INVITE, 200 OK, BYE, MESSAGE, SUBSCRIBE, NOTIFY; SDP bodies for codec/QoS
0xB115 — LTE ML1 serving cell	PHY	RSRP, RSRQ, SINR, PCI, EARFCN, timing advance
0xB97F — NR L1 meas	PHY	NR SS-RSRP, SS-RSRQ, SS-SINR, beam index, SSB index
Event 0x1BE4 — Call origination	Telephony	Call type, number, domain (CS/PS)
Event 0x1BE5 — Call connect	Telephony	Connection timestamp, domain
Event 0x1BE6 — Call end	Telephony	End cause, duration
0x1805 — EAP-AKA	Security	Authentication procedure for IMS/ePDG

Critical: For modem logs, you need a message dictionary / DM (Diagnostic Monitor) definition database to decode binary payloads. Options:

1. Use Qualcomm's QCAT/QXDM SDK if available under license to export to text/XML, then parse the text output.
2. Build a lightweight decoder for the highest-value messages listed above using publicly documented ASN.1 structures from 3GPP (36.331 for LTE RRC, 38.331 for NR RRC, 24.301 for EPS NAS, 24.501 for 5GS NAS).
3. Export from QXDM/QCAT to .txt or .xml format as a preprocessing step, then parse structured text.

Recommendation: Use option 3 (QXDM/QCAT text export) as the pragmatic first step, with option 2 as a long-term investment for a fully integrated pipeline. Wrap QCAT CLI in an automated export pipeline that produces one-message-per-line JSON or structured text.

3.4 Carrier Configuration Parsing

Parse `carrier_config_*.xml` and `CarrierConfig` dumps for per-operator overrides. Critical keys:

```
carrier_volte_available_bool
carrier_wfc_ims_available_bool
carrier_vt_available_bool
```

```
carrier_rcs_config_xml
ims.prefer_wfc_over_lte
carrier_nr_available_int_array
carrier_5g_standalone_available_bool
carrier_vonr_available_bool
use_iwlan_for_emergency_call_bool
carrier_supports_ss_over_ut_bool
carrier_esim_download_requires_wifi
ntn_connect_available_bool
```

These are essential because many issues stem from misconfigured carrier config overlays.

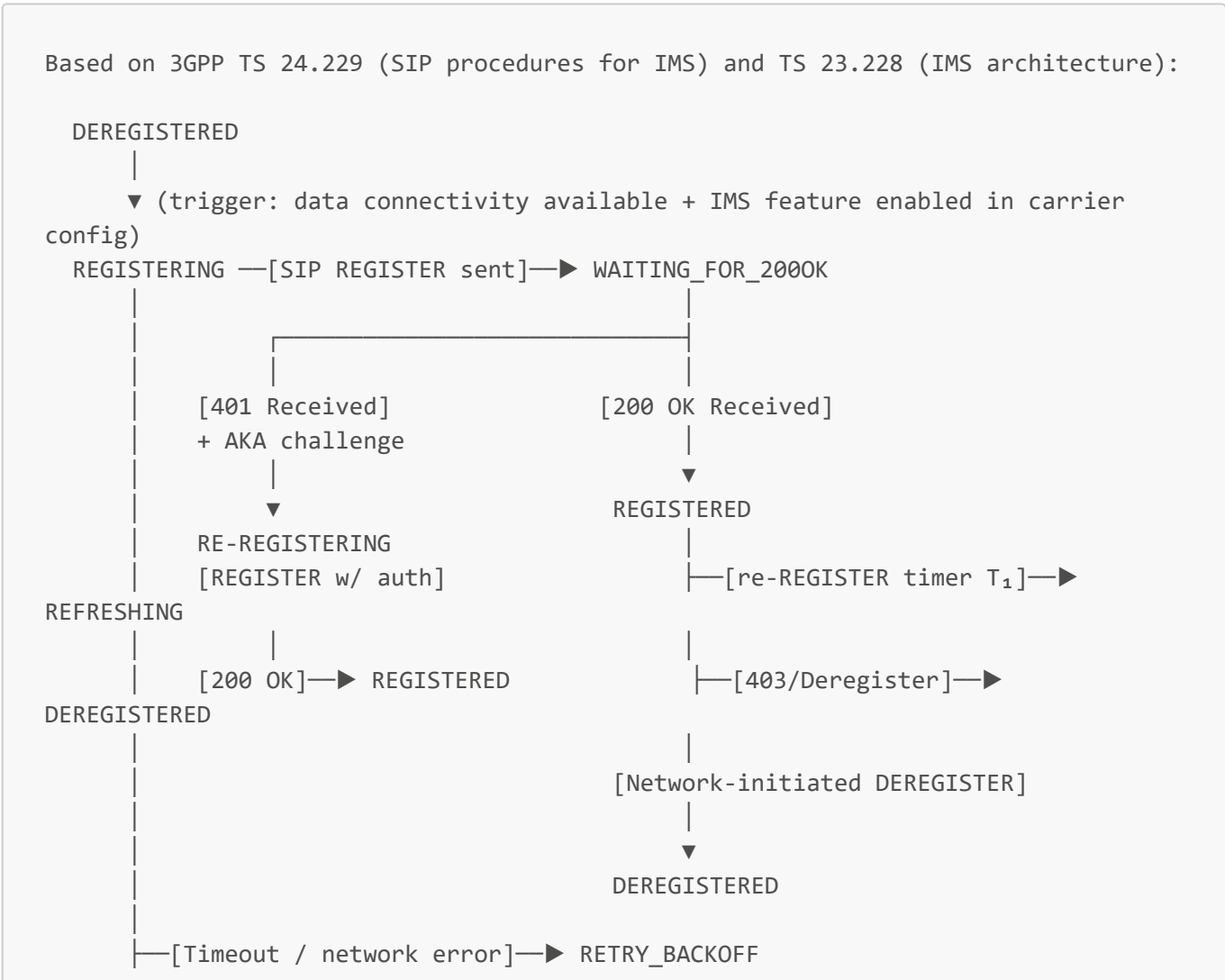
4. Domain Knowledge Layer — State Machines & Protocol FSMs

4.1 Why State Machines Are Critical

Raw log lines are noise without a model of expected behavior. The domain knowledge layer encodes "what *should* happen" so the anomaly detector can flag "what *did* happen differently."

4.2 Core State Machines to Implement

4.2.1 IMS Registration State Machine





Key anomaly signatures:

- REGISTER sent but no 401/200 within T_1 → network/IMS core unreachable
- 403 Forbidden → ISIM/AKA authentication failure, carrier config mismatch
- 503 Service Unavailable → IMS PDN up but P-CSCF overloaded
- Repeated REGISTERING→DEREGISTERED cycles → registration flapping (check cause code)
- Missing initial REGISTER after PDN setup → ImsService not started, feature disabled in CarrierConfig

4.2.2 VoLTE/VoNR MO Call Flow State Machine

Based on 3GPP TS 24.229 §6.1 (call initiation), TS 23.401/23.501 (bearer/QoS flow):

```

IDLE
|
▼ [user dials]
DIALING
|—[ImsPhone.dial() → ImsCallSession.start()]
|
▼
INITIATING
|—[SIP INVITE sent with SDP (codec, preconditions per RFC 3312)]
|—[Expect: Dedicated EPS bearer setup (QCI=1) for VoLTE]
|   or [QoS flow (5QI=1) for VoNR]
|
|—[100 Trying]—► INITIATING (stay)
|—[183 Session Progress + SDP answer]—► EARLY_MEDIA
|   |—[Dedicated bearer activated / QoS flow established]
|   |—[PRACK sent to confirm reliable provisional]
|   |—[Ring-back tone / early media via RTP]
|
|—[180 Ringing]—► ALERTING
|   |—[May or may not have early media]
|
|—[200 OK]—► CONNECTED
|   |—[ACK sent]
|   |—[RTP/RTCP media flowing on dedicated bearer/QoS flow]
|
|—[486 Busy / 603 Decline]—► ENDED (remote busy)
|—[408 Timeout]—► ENDED (network timeout)
|—[503 Service Unavailable]—► ENDED (IMS core issue)
|—[380 Alternative Service → CSFB]—► CS_FALLBACK
|—[Ooh no bearer setup]—► possible call drop
|
CONNECTED

```



```

|
|—[BYE from either end]—► DISCONNECTING—► ENDED
|—[RTP timeout (no media)]—► ENDED (media timeout / one-way audio)
|—[Ooh RRC release during call]—► ENDED (radio link failure)
|—[OHANDOVER (SRVCC / HO)]—► HANDOVER_IN_PROGRESS—► CONNECTED
|   |—[SRVCC: TS 23.216 – IMS to CS handover]
|   |—[LTE→NR or NR→LTE PS HO]
|   |—[LTE→WiFi ePDG HO for VoWiFi]

```

Key anomaly signatures:

- INVITE sent, no 100 Trying → P-CSCF not responding (check IMS PDN)
- 183 received but no dedicated bearer → EPS bearer activation rejected by network (check ESM cause)
- 380 Alternative Service → CSFB occurring indicates VoLTE call rejected, potential config/registration issue
- One-way audio (RTP flowing one direction only) → NAT/firewall issue, or incorrect SDP negotiation, or SRTP key mismatch
- SRVCC failure → missing SRVCC capability in UE or network, handover preparation failure in RRC

4.2.3 eSIM Profile Download State Machine

Based on GSMA SGP.22 (RSP Technical Specification) v3.x:

```

IDLE
|
▼ [user initiates download – activation code or SM-DP+ push]
LPA_INITIATED
|—[EuiccService.downloadSubscription()]
|—[LPA connects to SM-DP+ (HTTPS)]
|
▼
ES9p_INITIATED (Common Mutual Authentication)
|—[InitiateAuthentication → SM-DP+]
|—[SM-DP+ returns serverSigned1, serverCertificate, euiccCiPKIdToBeUsed]
|—[LPA forwards to eUICC → authenticateServer]
|—[eUICC validates SM-DP+ cert chain against CI PKI]
|
|—[eUICC auth failure]—► ERROR (CI mismatch, cert expired, unknown CI)
|
▼
MUTUAL_AUTH_COMPLETE
|—[AuthenticateClient → SM-DP+]
|—[SM-DP+ verifies eUICC cert and generates BPP]
|
|—[SM-DP+ rejects eUICC]—► ERROR (EID not whitelisted, capacity)
|
▼
BPP_DOWNLOAD
|—[GetBoundProfilePackage → SM-DP+]
|—[BPP (encrypted profile) received]

```

```

|—[LPA sends LoadBoundProfilePackage to eUICC]
|—[eUICC decrypts and installs profile]
|
|—[Installation error]—► ERROR (memory full, profile policy violation)
|
▼
PROFILE_INSTALLED (disabled state)
|
▼ [enable profile]
PROFILE_ENABLED
|—[UiccController detects new ICCID]
|—[SubscriptionManagerService creates new SubscriptionInfo]
|—[Telephony stack restarts with new profile]
|—[UICC → READY → LOADED → NAS Attach]

ERROR states → extract:
- ES9+ error codes (e.g., "8.1" = no matching EID, "8.8" = CI unknown)
- eUICC status codes from APDU responses
- HTTPS transport errors (TLS failures, DNS resolution)
- BPP decryption failures

```

4.2.4 EPS Attach & PDN Connectivity State Machine

Based on 3GPP TS 24.301 (NAS protocol for EPS):

```

EMM-DEREGISTERED
|
▼ [power on, SIM loaded, PLMN selected]
ATTACH_INITIATED
|—[Attach Request (EPS attach type, GUTI/IMSI, UE NW cap, PDN connectivity
req)]
|—[Identity Request?]—► [Identity Response (IMSI)]
|—[Authentication Request (RAND, AUTN)]—► [Authentication Response (RES)]
|—[Security Mode Command (EIA, EEA algorithms)]—► [Security Mode Complete]
|
|—[Attach Reject (EMM cause)]—► EMM-DEREGISTERED
|   Cause #3:  Illegal UE
|   Cause #6:  Illegal ME
|   Cause #7:  EPS services not allowed
|   Cause #8:  EPS and non-EPS services not allowed
|   Cause #11: PLMN not allowed
|   Cause #12: TA not allowed
|   Cause #13: Roaming not allowed
|   Cause #14: EPS services not allowed in this PLMN
|   Cause #15: No suitable cells
|   Cause #25: Not authorized for this CSG
|   Cause #40: No EPS bearer context activated
|
▼
EMM-REGISTERED
|—[Attach Accept (GUTI, TAI list, EPS bearer context activated)]

```

```

|—[Default EPS bearer context activated → PDN connected (usually IMS APN)]
|
|—[TAU procedure for mobility]
|—[Service Request for data/signalling]
|—[Extended Service Request for CSFB/VoLTE]
|
▼ [Additional PDN connectivity]
PDN_CONNECTIVITY_REQUEST
|—[PDN Connectivity Request (APN, PDN type, PCO)]
|—[Activate Default EPS Bearer Context Request from network]
|
|—[PDN Connectivity Reject (ESM cause)]
|   Cause #26: Insufficient resources
|   Cause #27: Missing or unknown APN
|   Cause #29: User authentication failed
|   Cause #30: Activation rejected by S-GW or P-GW
|   Cause #31: Request rejected, unspecified
|   Cause #32: Service option not supported
|   Cause #33: Service option not subscribed
|   Cause #50: PDN type IPv4 only allowed
|   Cause #51: PDN type IPv6 only allowed

```

4.2.5 5GS Registration & PDU Session State Machine

Based on 3GPP TS 24.501 (NAS protocol for 5GS):

```

5GMM-DEREGISTERED
|
▼
REGISTRATION_INITIATED
|—[Registration Request (registration type, 5GS mobile identity,
|   requested NSSAI, UE security capability)]
|—[Authentication procedures (5G-AKA or EAP-AKA')]
|—[Security Mode Command / Complete (NIA, NEA)]
|
|—[Registration Reject (5GMM cause)]
|   Cause #3: Illegal UE
|   Cause #11: PLMN not allowed
|   Cause #27: N1 mode not allowed
|   Cause #62: No network slicing subscription
|   Cause #69: Serving network not authorized for this SNPN
|   Cause #72: Non-3GPP access to CN not allowed
|   Cause #73: Temporarily not authorized for this SNPN
|   Cause #76: N2 not available for UPF for the PDU session
|
▼
5GMM-REGISTERED
|—[Registration Accept (5G-GUTI, TAI list, allowed NSSAI,
|   PDU session status, network slicing info)]
|
▼ [PDU session establishment]

```

```

PDU_SESSION_ESTABLISHMENT_REQUEST
|—[PDU Session Est. Request (PDU session ID, S-NSSAI, DNN, PDU type, SSC
mode)]
|—[PDU Session Est. Accept (QoS rules, authorized QoS flow descriptions,
session AMBR, PDU address, UP security)]
|
|—[PDU Session Est. Reject (5GSM cause)]
|   Cause #26: Insufficient resources
|   Cause #27: Missing or unknown DNN
|   Cause #28: Unknown PDU session type
|   Cause #29: User authentication or authorization failed
|   Cause #31: Request rejected, unspecified
|   Cause #67: Insufficient resources for specific slice and DNN

```

4.2.6 NTN (Non-Terrestrial Network) / Satellite Connectivity State Machine

Based on 3GPP TS 23.256 (NTN architecture), TS 38.300 (NR-NTN),
and Android SatelliteManager framework (android.telephony.satellite):

```

SATELLITE_MODEM_OFF
|
|▼ [SatelliteController.requestSatelliteEnabled(true)]
SATELLITE_MODEM_TURNING_ON
|—[Modem powers satellite radio]
|—[SatelliteModemInterface.setSatelliteEnabled()]
|
|—[Error: SATELLITE_NOT_SUPPORTED / ERROR_MODEM]—► OFF (modem doesn't
support NTN)
|
|▼
SATELLITE_MODEM_ON (IDLE)
|—[Satellite not visible / pointing required]
|   |—[PointingAppController.startPointingUI()]
|   |—[User aligns device with satellite]
|   └─► WAITING_FOR_CONNECTION
|
|—[Satellite visible / auto-connect (LEO constellation)]
|   └─► SATELLITE_CONNECTED
|
SATELLITE_CONNECTED
|—[NTN-specific NAS registration if full 3GPP NTN mode]
|—[Or proprietary satellite protocol for SOS/messaging-only mode]
|
|—[DatagramController.sendDatagram()]—► SENDING_DATAGRAM
|   |—[Success]—► DATAGRAM_SENT
|   |—[Failure: NETWORK_ERROR / MODEM_TIMEOUT]—► SEND_FAILED
|
|—[DatagramController.pollPendingDatagrams()]—► RECEIVING_DATAGRAM
|   |—[Datagram received callback]—► DATAGRAM_RECEIVED

```

```

|—[Signal lost / satellite not visible]—► SATELLITE_NOT_CONNECTED
|   |—[Retry / re-pointing required]
|

```

KEY ANOMALIES:

- Modem reports NTN capable but SatelliteController says NOT_SUPPORTED
 - carrier config "ntn_connect_available" not set
- Pointing UI never completes → device sensors/compass issue, or satellite orbit timing
- Datagram send timeout → link budget insufficient, UE power class issue, excessive propagation delay (GEO vs LEO RTT differences)
- Registration failure on NTN → TS 24.501 NTN-specific causes, TA update challenges with moving cells (satellite ephemeris)

4.2.7 VoWiFi / ePDG Tunnel State Machine

Based on 3GPP TS 23.402 (non-3GPP access), TS 24.302 (IKEv2 for ePDG):

```

IWLAN_IDLE
|
▼ [WiFi connected + WFC enabled in settings + carrier_wfc_ims_available=true]
IWLAN_HANDOVER_POLICY_CHECK
|—[AccessNetworksManager evaluates QualifiedNetworkService / ANE]
|—[IWLAN preferred? (roaming state, carrier policy, WiFi quality)]
|
|—[IWLAN not preferred]—► IWLAN_IDLE (stay on cellular)
|
▼
ePDG_CONNECTING
|—[EpdgTunnelManager.bringUpTunnel()]
|—[DNS resolution for ePDG FQDN:
   epdg.epc.mnc<MNC>.mcc<MCC>.pub.3gppnetwork.org]
|—[IKEv2 INIT exchange]
|—[IKEv2 AUTH with EAP-AKA' (TS 33.402)]
|
|—[DNS failure]—► ERROR (ePDG FQDN not resolving – incorrect PLMN, DNS
config)
|—[IKE timeout]—► ERROR (ePDG unreachable – firewall, NAT issue)
|—[EAP-AKA' failure]—► ERROR (auth failure – SIM/ISIM issue, AAA server)
|—[IKE AUTH rejected (AUTHENTICATION_FAILED)]—► ERROR
|—[IKE NOTIFY: NO_PROPOSAL_CHOSEN]—► ERROR (crypto mismatch)
|
▼
ePDG_CONNECTED (IPsec tunnel up)
|—[Inner IP address assigned]
|—[IMS re-registration over IWLAN]
|—[SIP REGISTER via IPsec tunnel → P-CSCF]
|
|—[WiFi disconnect]—► IWLAN_DISCONNECTING → HO back to cellular
|—[Dead Peer Detection failure]—► TUNNEL_DOWN → reconnect or HO
|—[ePDG-initiated rekey / delete]—► TUNNEL_DOWN

```

4.2.8 RCS (Rich Communication Services) State Machine

Based on GSMA RCC.07 (RCS UP specification), OMA SIMPLE (presence), 3GPP TS 24.229 (SIP for IMS):



KEY ANOMALIES:

- RCS feature tags missing from IMS registration
 - carrier config doesn't include RCS tags, or ImsRcsManager not bound
- OPTIONS always timeout → remote party on different RCS backend (interop issue)
- MSRP connection failure → port/NAT traversal issue, or OMACP not applied
- File transfer HTTP 413/507 → file too large / storage quota exceeded

5. Anomaly Detection & Context Builder

5.1 Rule-Based Anomaly Detection (Pre-LLM Filtering)

Before invoking the LLM, a deterministic anomaly detection layer should flag issues and reduce the log volume by 90–99%. This is essential for cost/latency.

Detection categories:

A. Error & Rejection Code Detection

```
# Pseudocode structure
ANOMALY_RULES = {
  "NAS_REJECT": {
    "pattern": r"(Attach Reject|Registration Reject|TAU Reject|PDN
Connectivity Reject|PDU Session.*Reject)",
    "extract": "cause_code",
    "severity": "CRITICAL",
    "spec_ref": "TS 24.301 §9.9.3.9 / TS 24.501 §9.11.3.2",
    "context_window": "±30 seconds of surrounding logs"
  },
  "SIP_ERROR": {
    "pattern": r"SIP/2\.\0 [4-6]\d{2}",
    "extract": "status_code, reason_phrase, call_id",
    "severity": "HIGH",
    "spec_ref": "TS 24.229, RFC 3261",
    "context_window": "full SIP dialog (same Call-ID)"
  },
  "IKE_AUTH_FAILURE": {
    "pattern": r"(AUTHENTICATION_FAILED|NO_PROPOSAL_CHOSEN|TS_UNACCEPTABLE)",
    "extract": "IKE notify type",
    "severity": "HIGH",
    "spec_ref": "TS 24.302, RFC 7296"
  },
  "RRC_RELEASE_UNEXPECTED": {
    "pattern": r"RRCConnectionRelease|RRCRelease",
    "extract": "release cause",
    "severity": "MEDIUM",
    "condition": "during_active_call or during_data_transfer"
  },
  "MODEM_CRASH": {
    "pattern": r"(subsystem_restart|SSR|modem crashed|err_fatal)",
    "extract": "crash signature, timestamp",
    "severity": "CRITICAL",
    "context_window": "±60 seconds, include kernel/dmesg"
  },
  "ESIM_ERROR": {
    "pattern": r"
(EuiccService.*error|BPP.*fail|APDU.*error|ES9\+.*error|smdp.*error)",
    "extract": "ES9+ error code, APDU SW1/SW2, HTTP status",
    "severity": "HIGH",
```

```
    "spec_ref": "GSMA SGP.22"
  }
}
```

B. State Machine Violation Detection

For each FSM defined in Section 4, track the current state from log events and flag any transition that doesn't exist in the model:

```
IF observed_transition NOT IN allowed_transitions[current_state]:
    flag ANOMALY(
        type="STATE_VIOLATION",
        fsm=<fsm_name>,
        expected_states=allowed_transitions[current_state],
        observed_next=<observed_state>,
        triggering_event=<log_line>
    )
```

C. Timer & Timeout Violations

Timer	Spec Reference	Expected Value	Anomaly If
T3410 (Attach)	TS 24.301 §5.5.1.2.6	15 s	Attach Request with no response
T3510 (TAU)	TS 24.301 §5.5.3.2.6	15 s	TAU Request with no response
T3517 (Service Req)	TS 24.301 §5.6.1.6	5 s	Service Request no response
T3512 (Periodic TAU)	TS 24.301 §5.5.3.2.7	Network-assigned (GPRS Timer 3)	Overdue periodic TAU
T3550 (PDN/PDU)	TS 24.301/24.501	6 s	Bearer activation no response
SIP INVITE Timer B	RFC 3261 §17.1.1.2	32 s (64*T1)	No provisional response to INVITE
SIP re-REGISTER	TS 24.229 §5.1.1.4	expires/2 or network-assigned	Late re-registration causing gap
RRC T304 (HO)	TS 36.331 §5.3.5.4	varies (50ms–8s)	Handover failure on timeout
IKEv2 retransmit	RFC 7296 §2.4	exponential backoff 0.5–60s	ePDG tunnel setup timeout

D. Performance Threshold Violations

- RSRP < -120 dBm (LTE) or SS-RSRP < -120 dBm (NR) → coverage issue
- SINR < 0 dB → interference issue
- BLER > 10% sustained → poor radio conditions
- RLC retransmission rate > 15% → link quality degradation
- RTP jitter > 50ms or packet loss > 3% → VoLTE quality issue
- HARQ residual BLER > 2% → throughput degradation
- TCP retransmissions > 5% → transport layer issue

E. Battery Drain Anomaly Detection

Flag if:

- Any single UID holds a wakelock > 60 seconds continuously (partial_wake_lock)
- Any UID fires > 100 wakeup alarms per hour
- Modem stays in CONNECTED mode > expected duration after data transfer ends
- WiFi scan frequency > 4 scans/minute when screen off
- RIL requests cycling (repeated setupDataCall/deactivateDataCall)
- IMS re-registration looping (> 5 REGISTER attempts per hour)
- RRC Connected→Idle transitions delayed (inactivity timer misconfigured)

5.2 Cross-Layer Correlation Engine

This is the most valuable non-LLM component. It links events across layers:

Example correlation: "VoLTE call drop"

Application layer: `ImsCallSession.terminate(CAUSE_NETWORK_ERROR)`

↑ correlate via `call_id`

SIP layer: `BYE received or RTP timeout detected`

↑ correlate via `timestamp ±2s`

NAS layer: `Service Request or TAU during call?`

↑ correlate via `timestamp`

RRC layer: `RRC Release with cause "other" during active call`

↑ correlate via `timestamp`

L2/L1: `BLER spike, RSRP drop, measurement report triggering HO?`

↑ correlate via `timestamp`

Kernel: `Any modem SSR? Thermal throttling?`

Result: Linked evidence chain from radio conditions → RRC → NAS → SIP → app

Correlation keys:

- **Time-based:** ±N second window (configurable per layer pair)
- **Session-based:** `call_id` (SIP), `bearer_id` (EPS/5GS), `transaction_id` (RIL), eUICC operation handle
- **Entity-based:** `phone_id/slot_id` (for DSDS), `subscription_id`, APN/DNN

5.3 Context Packet Builder

The output of this layer is a structured **"evidence packet"** ready for LLM consumption:

```
{
  "issue_summary": {
    "category": "VOLTE_CALL_DROP",
    "timestamp_range": "2025-03-15T10:23:45Z to 2025-03-15T10:24:02Z",
    "duration_seconds": 17,
    "device": "Pixel 9 Pro, Android 15, QPR2",
    "carrier": "T-Mobile US (MCC=310, MNC=260)",
    "rat_at_issue": "NR-NSA (EN-DC: LTE B66 + NR n41)"
  },
  "anomalies_detected": [
    {
      "type": "RRC_RELEASE_DURING_CALL",
      "timestamp": "2025-03-15T10:23:58Z",
      "details": "RRCConnectionRelease received with cause='other' while in CONNECTED_DCH",
      "spec_ref": "TS 36.331 §5.3.8",
      "severity": "CRITICAL"
    },
    {
      "type": "RSRP_DROP",
      "timestamp": "2025-03-15T10:23:55Z",
      "details": "LTE RSRP dropped from -95 dBm to -122 dBm within 3 seconds",
      "preceding_measurement_reports": ["MeasReport at T-5s: A3 event triggered for PCI 247"]
    }
  ],
  "correlated_event_chain": [
    {"layer": "L1/PHY", "time": "T-5s", "event": "RSRP degradation, A3 event triggered"},
    {"layer": "RRC", "time": "T-3s", "event": "MeasurementReport sent (PCI 247, RSRP -88)"},
    {"layer": "RRC", "time": "T-2s", "event": "RRCConnectionReconfiguration (HO command to PCI 247)"},
    {"layer": "RRC", "time": "T-0.5s", "event": "RACH attempt on target cell - FAILURE (T304 expired)"},
    {"layer": "RRC", "time": "T+0s", "event": "RRC Connection Re-establishment attempt"},
    {"layer": "RRC", "time": "T+2s", "event": "Re-establishment REJECT → RRC release"},
    {"layer": "NAS", "time": "T+2.5s", "event": "NAS: Local detach, re-attach triggered"},
    {"layer": "SIP", "time": "T+3s", "event": "IMS registration lost, call session terminated"},
    {"layer": "APP", "time": "T+4s", "event": "ImsPhone: onCallTerminated(CAUSE=ERROR_UNSPECIFIED)"}
  ],
  "relevant_config": {
    "carrier_volte_available": true,
    "carrier_vonr_available": false,
    "srvcc_supported": true,
    "wfc_mode": "wifi_preferred"
  }
}
```

```

    },
    "radio_environment": {
      "serving_cell": "PCI 312, EARFCN 66986, RSRP -95→-122, RSRQ -12→-18",
      "nr_scg": "PCI 510, NR-ARFCN 520110, SS-RSRP -100",
      "neighbor_cells": [
        "PCI 247, RSRP -88 (HO target, failed)",
        "PCI 155, RSRP -105"
      ]
    },
    "raw_log_excerpts": {
      "rrc_messages": ["<base64 or decoded RRC messages>"],
      "sip_messages": ["<SIP INVITE...>", "<SIP BYE...>"],
      "logcat_relevant": ["<filtered logcat lines for the time window>"]
    }
  }
}

```

6. LLM Reasoning Layer — Prompt Engineering & Architecture

6.1 System Prompt Architecture

Use domain-specific system prompts that encode the "expert persona" and analysis methodology:

SYSTEM PROMPT TEMPLATE (example: VoLTE Call Analysis):

You are an expert VoLTE/VoNR/VoWiFi call analysis engine with deep knowledge of:

SPECIFICATIONS:

- 3GPP TS 24.229 (SIP procedures for IMS)
- 3GPP TS 23.228 (IMS architecture)
- 3GPP TS 23.401 (EPS system architecture – bearer management)
- 3GPP TS 23.501/23.502 (5GS architecture – QoS flows)
- 3GPP TS 36.331 (LTE RRC) and TS 38.331 (NR RRC)
- 3GPP TS 24.301 (EPS NAS) and TS 24.501 (5GS NAS)
- 3GPP TS 23.216 (SRVCC – Single Radio VCC)
- 3GPP TS 26.114 (MTSI – Media Handling and Interaction)
- RFC 3261 (SIP), RFC 3312 (preconditions), RFC 4566 (SDP)

ANDROID IMPLEMENTATION:

- ImsPhone, ImsCall, ImsCallSession (frameworks/opt/telephony)
- GsmCdmaPhone call tracking and location updates
- TelephonyConnection for Ooh Telecom framework integration
- ImsService / MmTelFeature / RcsFeature HAL binding
- QualifiedNetworksService for IWLAN/cellular preference
- DataNetwork and DataNetworkController for bearer management

ANALYSIS METHODOLOGY:

1. First, identify the CATEGORY of the issue from the evidence packet (call setup failure, mid-call drop, one-way audio, OHDROP in HO, SRVCC failure, ohedge audio quality, ohedge ohdrop after OHANDOVER)
2. Examine the correlated event chain BOTTOM-UP (radio → RRC → NAS → SIP → app)

- 3. Identify the FIRST divergence from the expected call flow
- 4. Map the divergence to a specific root cause using:
 - a. Error codes / cause values → look up in the relevant spec
 - b. Timer expirations → identify which timer and why it expired
 - c. Missing messages → what was expected but not seen?
 - d. State machine violation → which transition was illegal?
- 5. Assess whether the issue is:
 - a. NETWORK-SIDE (core network, RAN, transport)
 - b. DEVICE-SIDE (modem firmware, ohedge framework, ohedge carrier config)
 - c. ENVIRONMENTAL (radio conditions, interference, mobility)
- 6. Provide the root cause with SPEC REFERENCES and a confidence level
- 7. Suggest next debugging steps or fixes

OUTPUT FORMAT:

```
{
  "root_cause_summary": "<one-line summary>",
  "detailed_analysis": "<step-by-step reasoning>",
  "root_cause_category":
    "NETWORK|DEVICE_MODEM|DEVICE_FRAMEWORK|DEVICE_CONFIG|ENVIRONMENTAL",
  "spec_references": ["<TS/RFC references with clause numbers>"],
  "confidence": "HIGH|MEDIUM|LOW",
  "confidence_rationale": "<why this confidence level>",
  "suggested_actions": ["<ordered list of next steps>"],
  "additional_logs_needed": ["<if confidence is LOW, what additional data would
help>"]
}
```

6.2 RAG (Retrieval-Augmented Generation) Architecture

Even though the LLM is pre-trained on telecom specs, RAG is essential for:

RAG Knowledge Bases to build:

Knowledge Base	Contents	Update Frequency
3GPP Spec Chunks	Chunked and indexed TS documents with clause-level granularity (focus on cause codes, procedures, timers, IEs)	Per 3GPP release cycle (quarterly)
GSMA PRDs	SGP.22, SGP.32 (eSIM), RCC.07 (RCS), IR.92 (VoLTE), IR.94 (ViLTE), IR.51 (VoNR), NG.114 (VoNR/EPS FB)	Per GSMA release
Known Issues DB	Historical bug reports with root causes, resolutions, and associated log patterns	Continuous (every resolved bug)
Carrier Config DB	Per-MCC/MNC carrier config values, known quirks, specific behavior	Per carrier update
Modem Release Notes	Qualcomm modem firmware release notes, known issues, fixed issues per build	Per modem build

Knowledge Base	Contents	Update Frequency
Android OASP/CTS	Android telephony CTS test expectations, ohedge ohedge TelephonyManager API contracts	Per Android release
OEM-Specific	OEM customization layers, vendor RIL quirks, proprietary extensions	As updated

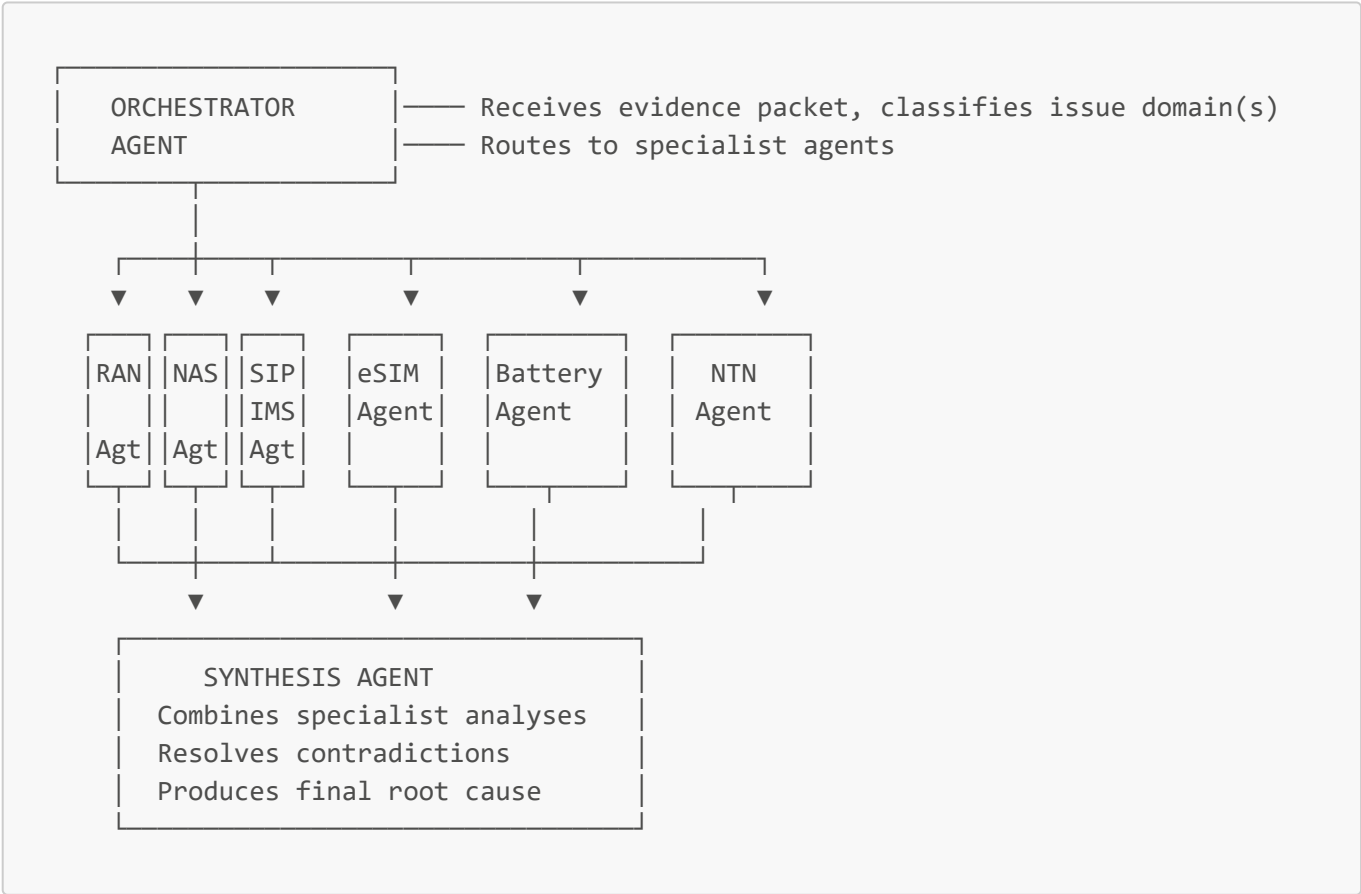
RAG retrieval strategy:

For each evidence packet:

1. Extract key entities: cause codes, message types, error strings, carrier ID
2. Query "Known Issues DB" first – if match with >0.85 similarity, fast-path answer
3. Query "3GPP Spec Chunks" for relevant procedure descriptions and cause code tables
4. Query "Carrier Config DB" for carrier-specific known behaviors
5. Inject top-K retrieved chunks into the LLM context along with the evidence packet

6.3 Multi-Agent Architecture (for Complex Issues)

For issues that span multiple domains, use a multi-agent approach:



Example multi-agent flow:

- Issue: "VoWiFi call drops when leaving WiFi coverage"
- Orchestrator routes to: WiFi/ePDG Agent + IMS/SIP Agent + RAN Agent + Data Agent
- WiFi/ePDG Agent: analyzes IPsec tunnel teardown timing
- IMS/SIP Agent: analyzes SIP re-registration handover
- RAN Agent: analyzes LTE re-acquisition timing
- Data Agent: analyzes PDN handover from IWLAN to LTE
- Synthesis Agent: correlates that ePDG tunnel was released *before* LTE PDN was established → gap in IMS bearer → call dropped. Root cause: **AccessNetworksManager** handover policy timing issue.

7. Specific Analysis Modules — Detailed Specifications

7.1 eSIM / eUICC Analysis Module

Input signals:

- **EuiccController**, **EuiccService**, **EuiccCardManager** logcat tags
- APDU command/response logs (CLA INS P1 P2 Lc Data Le → SW1 SW2)
- ES9+ HTTPS traffic logs (to SM-DP+)
- **UiccController** state transitions

What the LLM should analyze:

1. **Profile download failures:** Map ES9+ function error codes to SGP.22 §5.6 (error handling). Example: **SubjectCodeEnum** + **ReasonCodeEnum** → human-readable explanation + root cause.
2. **eUICC certificate chain validation:** If **authenticateServer** fails, check:
 - CI (Certificate Issuer) PKI mismatch — is the eUICC using GSMA CI or a test CI?
 - Certificate expiry
 - CRL (Certificate Revocation List) check failure
3. **Profile enable/disable issues:**
 - PPR (Profile Policy Rules) blocking operations (SGP.22 §3.2)
 - Concurrent profile limits
 - MEP (Multiple Enabled Profiles) vs single-enabled logic
4. **UICC state machine after profile switch:**
 - **UiccController**: ABSENT → PRESENT → READY → LOADED
 - **SubscriptionManagerService**: new SubscriptionInfo creation
 - NAS attach with new IMSI from profile

LLM prompt enrichment for eSIM:

Include in context:

- SGP.22 error code tables (§2.5.6: SubjectCode + ReasonCode matrix)
- SGP.22 ES9+ function definitions (§5.6 – InitiateAuthentication through HandleNotification)
- Device EID, eUICC firmware version (if available)

- SM-DP+ address from activation code
- CI PKI identifier (GSMA production vs test)

7.2 Battery Drain Analysis Module

Input signals:

- `dumpsys batterystats --checkin` (machine-readable format)
- `dumpsys batterystats` (human-readable)
- `dumpsys alarm`
- `dumpsys jobscheduler`
- Logcat tags: `PowerManagerService`, `BatteryStats`, `AlarmManager`, `WakeLock`
- Kernel wakelocks from `/proc/wakelocks` or `wakeup_sources`

Analysis framework:

1. IDENTIFY drain period:
 - Compare screen-on vs screen-off drain rates
 - Normal screen-off: <2% per hour for modern devices
 - Flag if screen-off drain > 3% per hour
2. ATTRIBUTE drain to components:
 - a. MODEM drain:
 - Excessive RRC Connected time (check: `DataConnection` state logs)
 - Registration looping (PLMN search cycling)
 - IMS re-registration looping
 - Cell reselection storms (weak coverage, ping-pong)
 - NR SA/NSA hunting when 5G is marginal
 - b. WIFI drain:
 - Excessive WiFi scans (check `WifiNative` scan logs)
 - WiFi disconnect/reconnect cycling
 - PNO (Preferred Network Offload) scan frequency
 - c. APPLICATION drain:
 - Wakelock holders (top 10 by duration)
 - Alarm frequency per UID
 - Network usage per UID during screen-off
 - GPS usage per UID
 - SyncManager: excessive syncs
 - d. FRAMEWORK drain:
 - JobScheduler: jobs running too frequently
 - ConnectivityService: network validation loops
 - TelephonyRegistry: excessive broadcasts
3. CROSS-REFERENCE with connectivity issues:
 - Battery drain is often a SYMPTOM of connectivity issues
 - Example: modem can't find suitable cell → continuous PLMN search → battery drain

- Example: IMS registration failing → retry → drain
- Example: data stall detection → network re-evaluation → reconnect → drain

7.3 NTN/Satellite Connectivity Analysis Module

Input signals:

- `SatelliteController`, `SatelliteModemInterface`, `SatelliteSessionController`, `DatagramController`, `PointingAppController` logcat tags
- Modem NTN-specific logs (if available through Qualcomm QXDM — look for NTN-specific log packets if supported in modem build)
- `dumpsys telephony.registry` for satellite modem state

Analysis areas:

1. Satellite modem enable/disable issues:

- `SatelliteController.requestSatelliteEnabled()` result codes
- Carrier config: `satellite_attach_supported_bool`, `satellite_connection_supported_int_array`
- HAL binding: `SatelliteModemInterface` → vendor AIDL/HIDL satellite HAL

2. Satellite visibility and pointing:

- `PointingAppController` → device orientation sensor data
- Satellite ephemeris data freshness (TLE / almanac age)
- For LEO constellations (e.g., AST SpaceMobile, Lynk): pass timing and duration
- For GEO-based (e.g., proprietary SOS): elevation angle sufficiency

3. Datagram send/receive for satellite messaging:

- `DatagramController.sendDatagram()` → success/failure
- Timeout analysis: NTN has much higher RTT than terrestrial
 - GEO: ~600ms one-way propagation
 - LEO: ~20–40ms propagation but intermittent visibility
- Datagram fragmentation and reassembly

4. NR-NTN specific (3GPP Release 17+):

- TS 38.300 / 38.331 NTN-specific procedures
- Timing advance handling for satellite (TA can be very large)
- HARQ disabling in NTN mode (some configurations)
- GNSS-based positioning for NTN cell ID

7.4 L2/L3/L4 Protocol Analysis Module

L2 (MAC/RLC/PDCP) Analysis:

From Qualcomm logs:

- MAC: DL/UL grants, HARQ ACK/NACK stats, BSR values, scheduling request failures

- RLC: AM retransmission counts, poll/status PDU frequency, RLC reset events
 - High RLC retx → radio link issue or interference
 - RLC MAX_RETX reached → potential RLF (Radio Link Failure)
- PDCP: Reordering timer expiry, duplicate detection, ROHC decompression failures
 - ROHC failures → header compression context desync → data corruption

Key analysis:

- Correlate L2 stats degradation with L1 measurements (RSRP/RSRQ/SINR)
- Check for DRX configuration issues (long DRX causing delayed scheduling)
- Check CA (Carrier Aggregation) activation/deactivation patterns
- For EN-DC/NR-DC: SCG failure information (TS 38.331 §5.7.3)

L3 (RRC/NAS) Analysis:

RRC Analysis:

- Connection setup success/failure rate
- Handover success/failure (A3/A5/B1 event triggering → HO command → RACH on target)
- RRC Connection Re-establishment causes
- Measurement configuration analysis: are meas gaps configured correctly for inter-freq/inter-RAT?
- SCell addition/removal for CA
- MCG/SCG configuration for dual connectivity
- T310/T311 timer events → Radio Link Failure detection and recovery

NAS Analysis:

- Already covered in state machines above
- Additional: ESM/5GSM message analysis for dedicated bearer/QoS flow operations
- Protocol Configuration Options (PCO) analysis: DNS, P-CSCF addresses, MTU
- Extended DRX / PSM configuration for IoT use cases

L4 (TCP/UDP/IP) Analysis:

From logcat and tcpdump (if available):

- TCP retransmissions, RTT, window scaling
- TLS handshake failures (certificate issues for IMS/eSIM HTTPS connections)
- DNS resolution failures (critical for ePDG FQDN, SM-DP+ resolution)
- MTU/fragmentation issues (especially on IPsec tunnels for VoWiFi)
- QUIC/HTTP3 behavior for modern applications
- IPsec ESP packet analysis for VoWiFi tunnel integrity

8. Fine-Tuning Strategy

8.1 Training Data Preparation

Sources for supervised fine-tuning:

Data Source	Label Strategy	Volume Target
Resolved bug reports with root cause	(log_pattern, root_cause, resolution) tuples	10,000+ resolved issues
QA-engineer-annotated log sessions	Expert labels on log segments	5,000+ annotated sessions
Synthetic scenarios from lab reproduction	Known-good and known-bad call flows from test setup	2,000+ synthetic scenarios
3GPP conformance test logs	Expected pass/fail outcomes	1,000+ test case logs
Carrier-specific known issues	Carrier, issue pattern, root cause	500+ per major carrier

Fine-tuning data format (per training example):

```
{
  "system": "<domain-specific system prompt from §6.1>",
  "user": "<evidence packet from §5.3 in structured JSON>",
  "assistant": "<expert analysis in output format from §6.1>"
}
```

8.2 Evaluation Metrics

Metric	Definition	Target
Root cause accuracy	Does the identified root cause match the expert-verified cause?	>80% for top-1, >95% for top-3
Category accuracy	Is the issue category (NETWORK/DEVICE/ENV) correct?	>90%
Spec reference accuracy	Are cited 3GPP/GSMA references correct and relevant?	>85%
Actionability	Are the suggested next steps useful and ranked correctly?	Human eval: >4.0/5.0
False positive rate	Does the system flag non-issues as issues?	<5%
Latency	Time from log ingestion to root cause output	<60s for single-issue, <5min for full log analysis

9. Implementation Roadmap

Phase 1: Foundation (Months 1–3)

- Build logcat parser with tag classification and entity extraction

- Build dumpstate section extractor
- Implement basic rule-based anomaly detection (error codes, crash detection)
- Create initial system prompts for VoLTE call analysis and data connectivity
- Set up RAG pipeline with 3GPP spec chunks + cause code tables
- Build simple web UI for log upload → analysis output

Phase 2: Protocol Depth (Months 3–6)

- Integrate Qualcomm modem log parsing (start with QCAT text export)
- Implement state machines for IMS registration, call flow, EPS/5GS attach
- Build cross-layer correlation engine
- Add eSIM analysis module with SGP.22 error code enrichment
- Add NAS/RRC protocol analysis
- Build Known Issues DB and begin populating from historical bugs
- Begin supervised fine-tuning with labeled bug data

Phase 3: Intelligence (Months 6–9)

- Implement multi-agent architecture for complex cross-domain issues
- Add NTN/satellite analysis module
- Add battery drain analysis pipeline
- Build L2 analysis (MAC/RLC/PDCP stats correlation)
- Enhance RAG with carrier config DB and modem release notes
- Deploy evaluation framework and begin measuring accuracy metrics
- Add VoWiFi/ePDG tunnel analysis

Phase 4: Scale & Refinement (Months 9–12)

- Build native Qualcomm binary decoder for highest-value message types
- Implement continuous learning: new resolved bugs → fine-tuning data → model update
- Add automated comparison with known-good "golden" log patterns
- Multi-device / DSDS analysis support
- Integration with bug tracking systems (auto-classify, auto-suggest component)
- Performance optimization: streaming analysis for live logcat, <30s latency target
- Advanced: predictive analysis (detect degradation before failure)

Phase 5: Advanced (12+ months)

- Live monitoring mode: stream logcat + QXDM → real-time anomaly dashboard
- Comparative analysis across multiple devices/carriers for fleet-wide issues
- Integration with network-side (PCAP, CDR) data for end-to-end correlation
- Automated test log analysis for CI/CD pipelines (CTS/GTS telephony tests)
- Natural language query interface ("Why did this VoLTE call drop?" against a log)

10. Technology Stack Recommendations

Component	Recommended Technologies
-----------	--------------------------

Component	Recommended Technologies
Log parsing	Python (fast prototyping), Rust (production parsers for binary modem logs)
Structured storage	Elasticsearch/OpenSearch (full-text + structured log search), ClickHouse (time-series analytics)
State machine engine	Python transitions library or custom FSM framework
Vector DB for RAG	Milvus, Pinecone, or pgvector (for spec chunks and known issues)
LLM serving	Your proprietary model (vLLM, TensorRT-LLM, or TGI for serving)
Orchestration	LangChain/LangGraph or custom agent framework
Frontend	React + Monaco editor (for log viewing with annotations)
Pipeline	Apache Kafka (real-time log streaming), Airflow (batch processing)
Evaluation	Custom eval harness + human-in-the-loop review tool

11. Key Design Principles

1. **LLM is the reasoner, not the parser.** Never feed raw multi-GB logs to the LLM. Parse, filter, detect anomalies deterministically, build a focused evidence packet, and let the LLM reason over structured evidence.
2. **Encode domain knowledge in code, not just prompts.** State machines, timer validations, and cause code lookups are better as deterministic code than LLM inference. The LLM's strength is in synthesis, correlation reasoning, and natural language explanation.
3. **Spec references are non-negotiable.** Every root cause must be traceable to a 3GPP/GSMA/RFC clause. This is what separates expert analysis from guessing.
4. **Cross-layer correlation is the differentiator.** Most issues are not single-layer. The system that can link PHY degradation → RRC failure → NAS re-attach → IMS deregistration → call drop into a single narrative wins.
5. **Carrier-awareness is critical.** The same device with the same modem behaves differently on different carriers due to carrier config overlays. The system must be carrier-aware.
6. **Feedback loop is essential.** Every analysis the LLM produces should be reviewed by engineers. Correct analyses become training data. Incorrect analyses become counter-examples. The system gets better with use.
7. **Start with the highest-value use case.** VoLTE/VoNR call drops are the most common escalation from QA. Start there, prove value, then expand.